

FIAP

**CP3 — PLATAFORMA DE RECOMENDAÇÃO PARA
STREAMING**

Neo4j + Python + FastAPI + Redis

Checkpoint 3

Desenvolvimento de uma API de recomendação personalizada

Amanda Barral - RM 569662
Gabriela Fernandes - RM 568972
Julia Furiate - RM 570547
Livia Santos - RM 573963
Mariana Martins - RM5 74076

2026

1. Implementação do banco de dados Neo4j

Para iniciar o desenvolvimento da plataforma de recomendação, foi configurado o ambiente do projeto utilizando **Docker** e **Neo4j**. O Neo4j foi utilizado como banco de dados orientado a grafos, permitindo representar não apenas os dados, mas também os relacionamentos entre usuários, filmes e gêneros.

Para o primeiro teste da estrutura, foi criado um usuário, um filme e um gênero. Em seguida, foram estabelecidos os relacionamentos necessários para representar o comportamento do usuário na plataforma:

- **WATCHED** — representa que o usuário assistiu ao filme;
- **RATED** — representa a avaliação dada pelo usuário ao filme;
- **HAS_GENRE** — representa o gênero ao qual o filme pertence.

O resultado pode ser visualizado no grafo abaixo.



Figura 2 – Estrutura do grafo com usuários, filmes, gêneros e relacionamentos

E o texto:

Após a criação dos dados de teste, foram estabelecidos os relacionamentos entre usuários, filmes e gêneros. O grafo demonstra os históricos de visualização

(WATCHED), as avaliações dos usuários (RATED) e a associação dos filmes aos seus respectivos gêneros (HAS_GENRE). Essa estrutura será utilizada posteriormente para a geração das recomendações personalizadas.

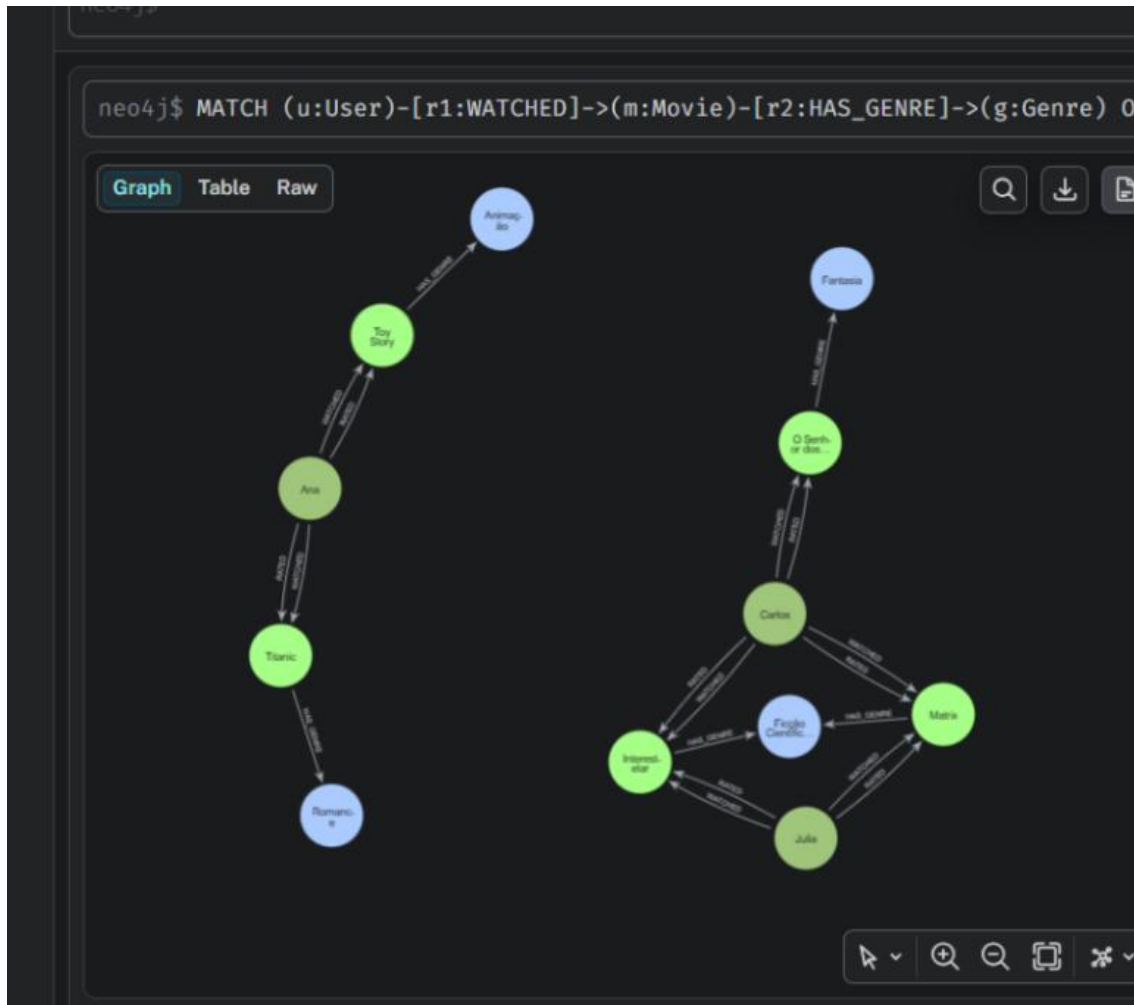
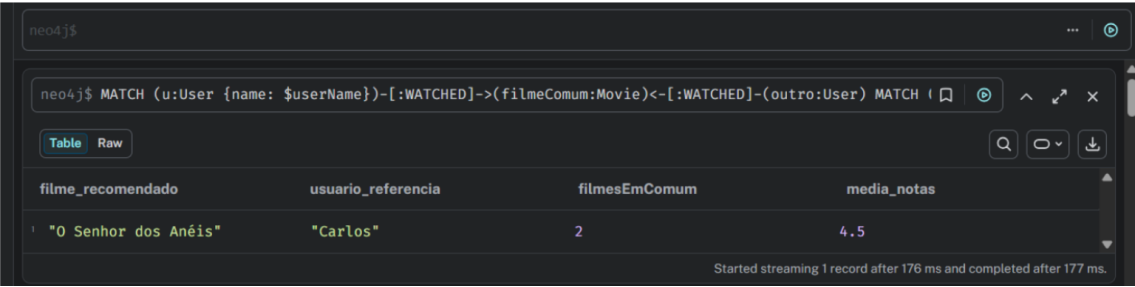


Figura 3 – Geração de recomendação no Neo4j

Texto abaixo da imagem:

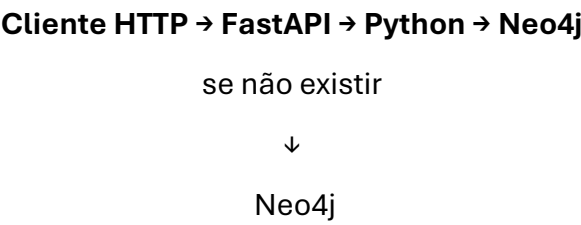
A consulta de recomendação identifica usuários com filmes assistidos em comum e utiliza seus históricos e avaliações para encontrar filmes que ainda não foram assistidos pelo usuário analisado. No teste realizado para Julia, o sistema identificou “O Senhor dos Anéis” como recomendação, com 2 filmes em comum

e média de notas 4,5.



2. Implementação da API com Python e FastAPI

Nesta etapa foi desenvolvida a camada de aplicação da plataforma, responsável por expor, por meio de uma **API REST**, as operações de cadastro, registro de comportamento e recomendação. A aplicação foi escrita em **Python** com o framework **FastAPI** e se comunica com o banco de grafos criado na Parte 1, seguindo o fluxo:



2.1 Tecnologias e bibliotecas

Biblioteca
FastAPI
Uvicorn
neo4j (driver oficial)
Pydantic e pydantic-settings
pytest e httpx

2.2 Estrutura da aplicação

Os arquivos da Parte 1 foram mantidos e a aplicação foi criada na pasta app/, com uma responsabilidade por arquivo:

Arquivo
app/main.py
app/config.py
app/database.py
app/schemas.py

Arquivo
app/repository.py
app/errors.py
app/routers/
tests/

2.3 Conexão com o Neo4j

O driver é criado uma única vez, quando a API é iniciada, e reaproveitado por todas as requisições; cada requisição recebe sua própria sessão, encerrada ao final. Os padrões de conexão são os mesmos do `docker-compose.yml` (`bolt://localhost:7687`, usuário `neo4j`, senha `senha123s`). O endpoint `GET /health` confirma a comunicação com o banco. Se o Neo4j estiver indisponível, a API responde **503** em poucos segundos, em vez de deixar a requisição pendurada.

2.4 Endpoints implementados

Método	Rota	Função
POST	<code>/users</code>	Cadastrar usuário
GET	<code>/users</code> • <code>/users/{nome}</code>	Listar e consultar usuários
POST	<code>/genres</code>	Cadastrar gênero
GET	<code>/genres</code>	Listar gêneros, com a quantidade de filmes
POST	<code>/movies</code>	Cadastrar filme, opcionalmente já ligado a gêneros existentes (HAS_GENRE)
GET	<code>/movies</code> • <code>/movies/{título}</code>	Listar e consultar filmes
POST	<code>/movies/{título}/genres</code>	Associar um gênero existente a um filme existente
POST	<code>/users/{nome}/watched</code>	Registrar filme assistido (WATCHED)
GET	<code>/users/{nome}/watched</code>	Listar filmes assistidos
POST	<code>/users/{nome}/ratings</code>	Registrar ou atualizar avaliação de 1 a 5 (RATED)
GET	<code>/users/{nome}/ratings</code>	Listar avaliações do usuário
GET	<code>/users/{nome}/recommendations</code>	Gerar recomendações personalizadas (parâmetro opcional <code>limit</code>)
GET	<code>/health</code>	Verificar a conexão com o Neo4j

2.5 Regras e validações

- **Sem duplicidade:** cadastrar um usuário, filme ou gênero que já existe retorna 409, em linha com as constraints da Parte 1.

- **Registros repetidos:** registrar duas vezes o mesmo filme assistido não cria um segundo relacionamento WATCHED (retorna 200).
- **Avaliações:** a nota deve estar entre 1 e 5 e só é possível avaliar um filme já assistido (caso contrário, 409). Avaliar de novo atualiza a nota, sem criar um segundo RATED.
- **Referências inexistentes:** usuário, filme ou gênero que não existem retornam 404 com mensagem explicativa.
- **Entrada inválida:** nomes e títulos vazios, com mais de 100 caracteres ou contendo “/” ou “\” retornam 422.

2.6 Recomendação personalizada

O endpoint GET /users/{nome}/recommendations recebe o usuário pela URL e o passa ao parâmetro \$userName da consulta da Parte 1. A aplicação lê o arquivo neo4j/recomendacao.cypher diretamente, sem cópia, de modo que existe uma única versão da consulta e a lógica não fica presa a nenhum usuário: qualquer usuário cadastrado pode solicitar recomendações, e um usuário inexistente retorna 404. Se um mesmo filme for sugerido por mais de um usuário de referência, ele aparece uma única vez, com a melhor referência.

A resposta traz o filme recomendado, o usuário de referência, a quantidade de filmes em comum e a média das notas:

```
GET /users/Julia/recommendations
```

```
{
  "user": "Julia",
  "total": 1,
  "recommendations": [
    {
      "filme_recomendado": "O Senhor dos Anéis",
      "usuario_referencia": "Carlos",
      "filmes_em_comum": 2,
      "media_notas": 4.5
    }
  ]
}
```

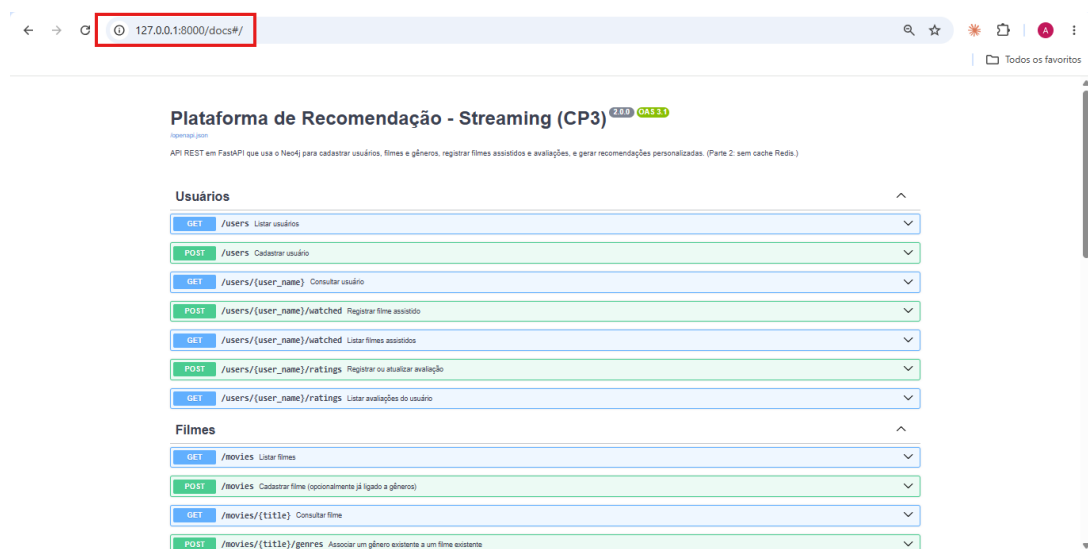


Figura 4a – Documentação interativa da API (Swagger): endpoints de usuários e filmes

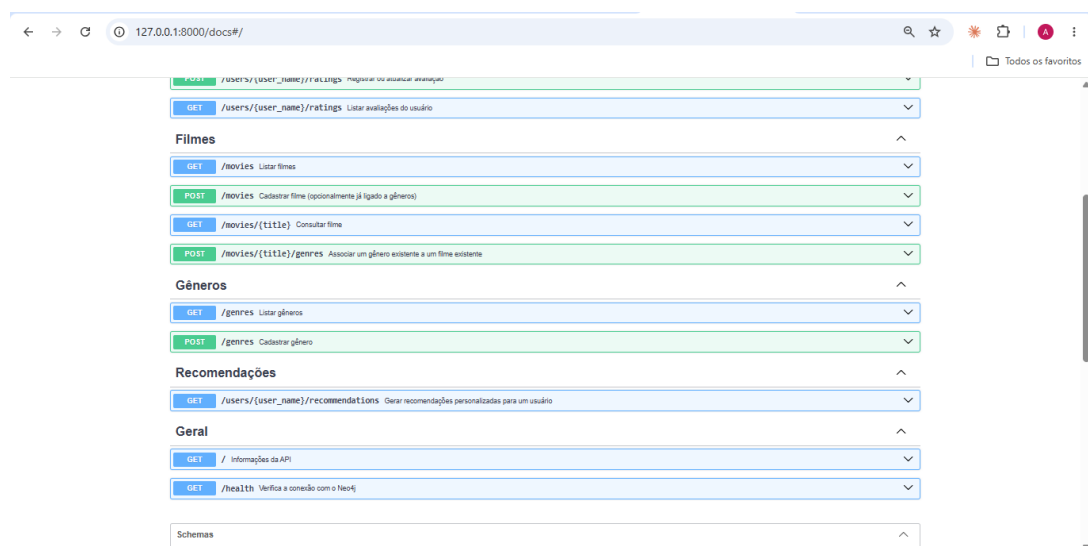


Figura 4b – Documentação interativa da API (Swagger): endpoints de gêneros, recomendações e verificação de saúde

A documentação gerada automaticamente pelo FastAPI lista todos os endpoints, agrupados por usuários, filmes, gêneros e recomendações, e permite testá-los diretamente pelo navegador.

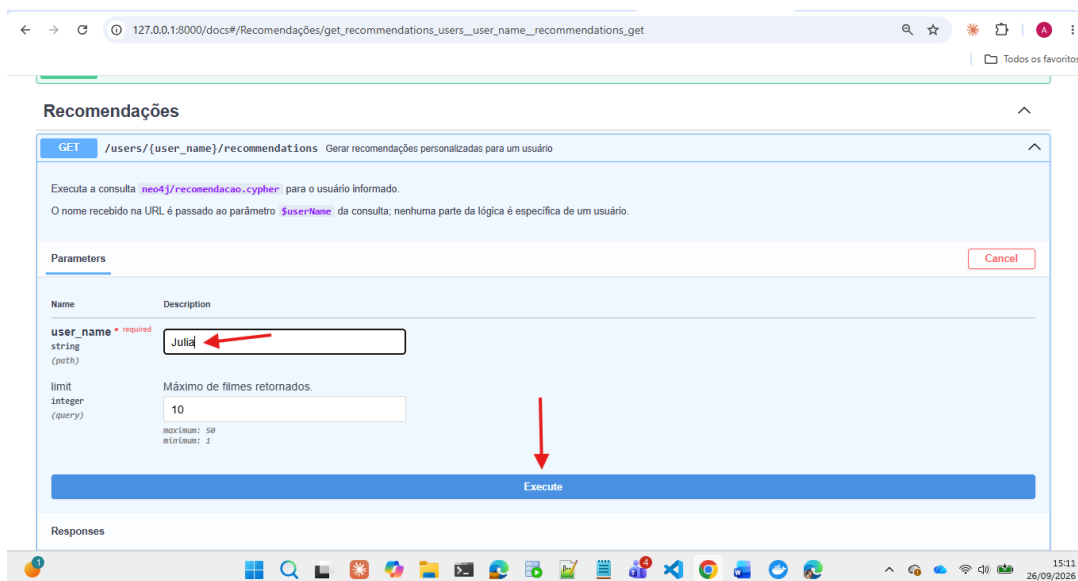


Figura 5a – Requisição de recomendação para Julia no Swagger

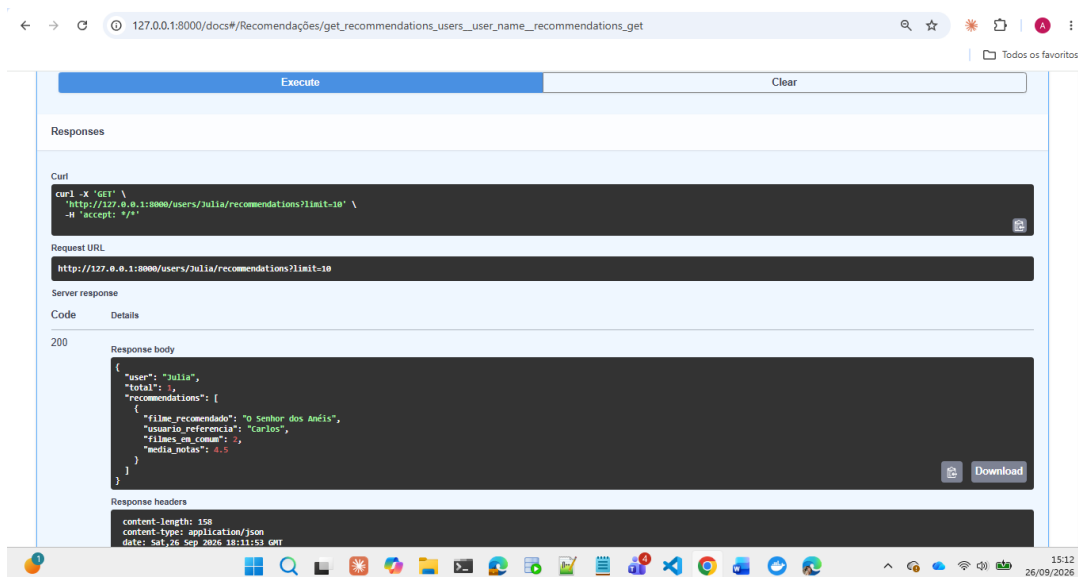


Figura 5b – Resposta da recomendação para Julia (código 200)

A requisição para Julia percorre FastAPI, Python e Neo4j e retorna “O Senhor dos Anéis”, com Carlos como referência, 2 filmes em comum e média de notas 4,5, resultado coerente com o obtido diretamente no Neo4j (Figura 3).

1. **Subir o Neo4j** (Neo4j Browser em <http://localhost:7474>, usuário neo4j, senha senha123s):

`docker compose up -d`

2. Criar o ambiente virtual e instalar as dependências:

```
python -m venv .venv
# Windows (PowerShell):
.venv\Scripts\Activate.ps1
# Linux/macOS:
source .venv/bin/activate
pip install -r requirements.txt
```

3. Iniciar a API:

```
uvicorn app.main:app --reload
```

4. **Acessar** <http://127.0.0.1:8000/docs> para testar os endpoints e <http://127.0.0.1:8000/health> para conferir a conexão com o Neo4j.

2.8 Testes

Foram criados dois níveis de teste:

- **Testes automatizados (pytest):** validam rotas, códigos de status (201, 200, 404, 409, 422), mensagens de erro, a passagem do usuário ao \$userName e o uso da consulta original da Parte 1. Não dependem do Neo4j. Execução: pytest.
- **Teste de ponta a ponta (smoke_test.py):** usa a API e o Neo4j reais e executa 31 verificações cobrindo cadastros, filmes assistidos, avaliações e recomendações, inclusive para um usuário diferente da Julia. Os dados temporários usam o prefixo “smoke_teste_” e são apagados ao final; os dados da Parte 1 não são alterados. Execução: python tests/smoke_test.py.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Jfox2026> cd Projetos\CP3-STREAMING
PS C:\Users\Jfox2026\Projetos\CP3-STREAMING> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Jfox2026\Projetos\CP3-STREAMING> python tests/smoke_test.py
Testando http://127.0.0.1:8000 (prefixo dos dados temporários: smoke_teste_b024e5)

1) Conexão
[OK] GET /health -> 200 (API conectada ao Neo4j)

2) Dados da Parte 1 e recomendação para a Julia
[OK] Usuários Julia, Carlos e Ana existem
[OK] GET /users/Julia/recommendations -> 200
[OK] Julia recebe 'O Senhor dos Anéis' (ref. Carlos, 2 em comum, média 4.5)

3) Cadastros (usuário, gênero, filme)
[OK] POST /users -> 201
[OK] POST /users duplicado -> 409
[OK] POST /genres -> 201
[OK] POST /genres duplicado -> 409
[OK] POST /movies com gênero -> 201
[OK] POST /movies duplicado -> 409
[OK] POST /movies com gênero inexistente -> 404
[OK] GET /movies/{titulo} -> 200
[OK] GET /genres mostra o gênero com 1 filme
[OK] POST /movies/Matrix/genres (já associado) -> 200, sem duplicar

4) Filmes assistidos
[OK] POST watched Interstellar -> 201
[OK] POST watched repetido -> 200 (sem duplicar)
[OK] POST watched do filme de teste -> 201
[OK] POST watched com filme inexistente -> 404
[OK] POST watched com usuário inexistente -> 404
[OK] GET watched lista exatamente 2 filmes
```

Figura 6a – Teste de ponta a ponta: conexão, dados da Parte 1, cadastros e filmes assistidos

```
Windows PowerShell

4) Filmes assistidos
[OK] POST watched Interstellar -> 201
[OK] POST watched repetido -> 200 (sem duplicar)
[OK] POST watched do filme de teste -> 201
[OK] POST watched com filme inexistente -> 404
[OK] POST watched com usuário inexistente -> 404
[OK] GET watched lista exatamente 2 filmes

5) Avaliações
[OK] Avaliar filme NÃO assistido -> 409
[OK] POST ratings -> 201
[OK] POST ratings de novo -> 200 (atualiza)
[OK] Apenas 1 avaliação, com a nota atualizada (3)
[OK] Nota 6 -> 422

6) Recomendação para outro usuário (prova que não está presa à Julia)
[OK] GET recommendations do usuário de teste -> 200
[OK] Recebe 'Matrix' e 'O Senhor dos Anéis' (assistiu Interstellar como Julia e Carlos)
[OK] Não recebe filmes que já assistiu
[OK] GET recommendations do Carlos -> 200
[OK] Parâmetro limit=1 devolve no máximo 1 filme
[OK] Recomendação para usuário inexistente -> 404

Dados temporários do teste removidos.

Resultado: 31/31 verificações OK

PARTE 2 VALIDADA: FastAPI -> Python -> Neo4j funcionando.
(.venv) PS C:\Users\Jfox2026\Projetos\CP3-STREAMING>
```

Figura 6b – Teste de ponta a ponta: avaliações, recomendações e resultado final

O resultado confirma que a API cadastra e consulta dados no Neo4j, registra WATCHED e RATED sem duplicar relacionamentos e gera recomendações para diferentes usuários, concluindo a comunicação FastAPI → Python → Neo4j. A integração com o Redis, com cache de recomendações, TTL e invalidação, será desenvolvida na Parte 3.

```
(.venv) PS C:\Users\Jfox2026\Projetos\CP3-STREAMING> pytest [100%]  
===== warnings summary =====  
.venv\Lib\site-packages\fastapi\testclient.py:1  
C:\Users\Jfox2026\Projetos\CP3-STREAMING\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.  
  from starlette.testclient import TestClient as TestClient # noqa  
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html  
37 passed, 1 warning in 0.29s  
(.venv) PS C:\Users\Jfox2026\Projetos\CP3-STREAMING> |
```

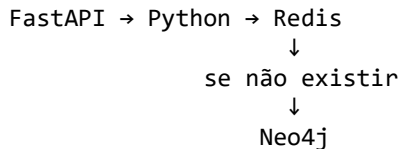
Figura 7 – Execução dos testes automatizados (pytest)

Os 37 testes automatizados são executados sem depender do Neo4j e cobrem rotas, códigos de status e regras de validação. O aviso exibido é um alerta de depreciação de uma biblioteca usada nos testes, sem efeito sobre o resultado.

3. Implementação do Redis e Cache

Nesta etapa foi realizada a integração do Redis com a API desenvolvida na Parte 2. O Redis é utilizado como camada de cache para armazenar temporariamente as recomendações já calculadas, evitando consultas repetidas ao Neo4j quando a mesma recomendação ainda está disponível no cache.

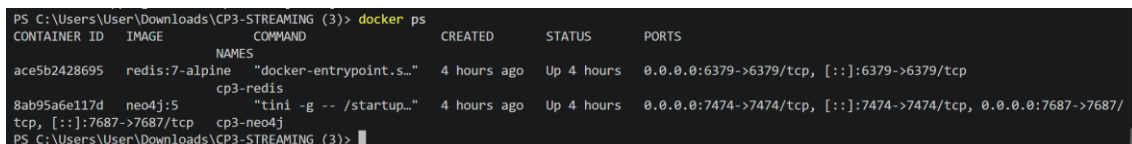
Com a integração, o fluxo de processamento passa a ser:



Quando a recomendação já está armazenada no Redis, a API pode retorná-la diretamente. Quando a chave não está disponível, a aplicação consulta o Neo4j, grava o resultado no Redis com TTL e retorna a recomendação. Esse comportamento caracteriza o padrão cache-aside.

3.1 Configuração do Redis

O Redis foi integrado ao projeto para funcionar junto ao Neo4j no ambiente da aplicação, ambos executados em containers Docker. A Figura 8 mostra a saída do comando `docker ps` com os dois containers em execução: `cp3-redis`, baseado na imagem `redis:7-alpine` e publicado na porta 6379, e `cp3-neo4j`, baseado na imagem `neo4j:5` e publicado nas portas 7474 (Neo4j Browser) e 7687 (Bolt). Ambos aparecem com status Up há 4 horas.



CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
ace5b2428695	redis:7-alpine	cp3-redis	"docker-entrypoint.s..."	4 hours ago	Up 4 hours	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
8ab95a6e117d	neo4j:5	cp3-neo4j	"tini -g -- /startup..."	4 hours ago	Up 4 hours	0.0.0.0:7474->7474/tcp, [::]:7474->7474/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp

Figura 8 — Containers `cp3-redis` (`redis:7-alpine`, porta 6379) e `cp3-neo4j` (`neo4j:5`) em execução, conforme o comando `docker ps`.

A aplicação possui uma configuração para ativar ou desativar o cache:

```
CACHE_ENABLED=true
```

O tempo de permanência das recomendações no Redis é configurado por:

```
CACHE_TTL_SECONDS=300
```

O valor de 300 segundos também é confirmado pela resposta do endpoint de estatísticas apresentada na Figura 13, em que o campo `ttl_seconds` aparece com valor 300, e pelo campo `ttl_remaining_seconds = 300` retornado na requisição com CACHE MISS da Figura 9.

3.2 Funcionamento do Cache

A rota utilizada para solicitar recomendações é:

```
GET /users/{nome}/recommendations?limit=10
```

A documentação da API descreve o fluxo cache-aside: primeiro a aplicação procura a recomendação no Redis; se encontrar, retorna o resultado e marca a resposta como CACHE HIT; se não encontrar, consulta o Neo4j, grava o resultado no Redis com TTL e marca a resposta como CACHE MISS.

Os testes anexados comprovam esse comportamento para a usuária Julia. Na Figura 9, a primeira requisição retorna cache = MISS, response_time_ms = 10 e ttl_remaining_seconds = 300. A recomendação retornada é "O Senhor dos Anéis", tendo Carlos como usuário de referência, com 2 filmes em comum e média de notas 4,5.

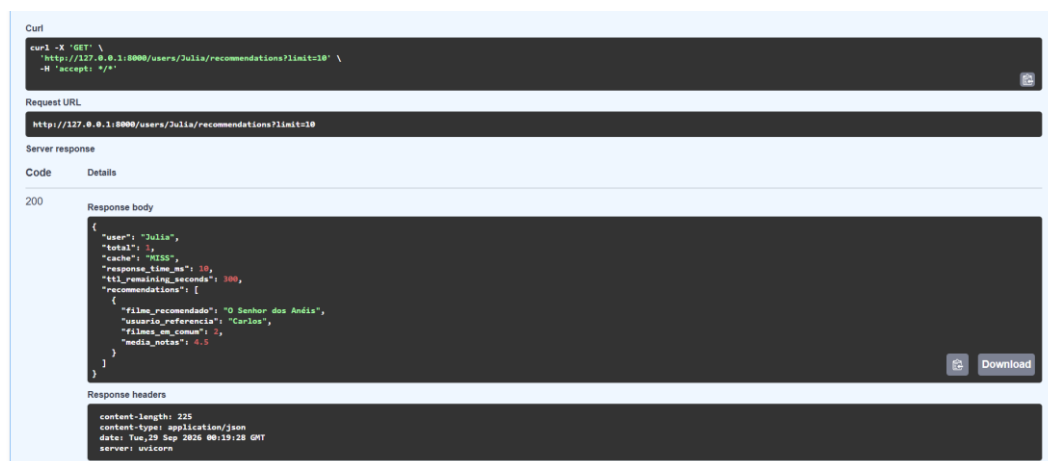


Figura 9 — Requisição de recomendação para Julia com CACHE MISS e tempo de resposta de 10 ms.

Após a recomendação ser armazenada, a mesma requisição é repetida. A Figura 10 mostra cache = HIT, response_time_ms = 1,64 e ttl_remaining_seconds = 295, mantendo a mesma recomendação retornada.

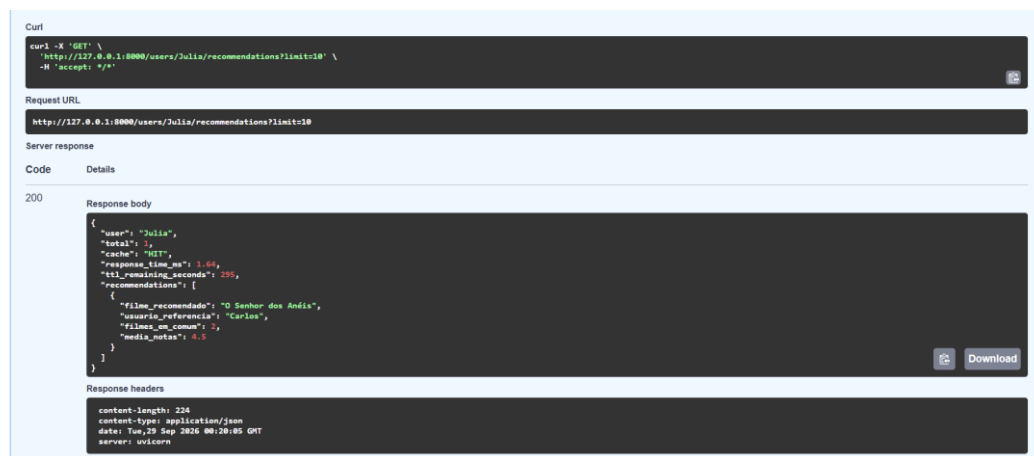


Figura 10 — Requisição subsequente para Julia com CACHE HIT, tempo de resposta de 1,64 ms e TTL restante de 295 segundos.

3.3 CACHE HIT e CACHE MISS

Os campos apresentados pela API permitem identificar diretamente o estado do cache. CACHE MISS indica que a recomendação não estava disponível no Redis

no momento da solicitação e que foi necessário obter o resultado por meio da consulta ao Neo4j. CACHE HIT indica que o resultado já estava disponível no Redis e pôde ser retornado pela camada de cache.

Nos testes anexados, a diferença está comprovada pelas duas requisições de Julia: a Figura 9 registra MISS e a Figura 10 registra HIT. A comparação dos tempos observados também mostra diferença entre as duas respostas: 10 ms no MISS e 1,64 ms no HIT.

O documento-base também descreve os estados DOWN, quando o Redis está indisponível, e DISABLED, quando o cache está desativado. O estado DISABLED está comprovado na Figura 15 (seção 4.1), em que a API retorna cache = DISABLED com o cache desativado. Não há, entre os prints anexados, uma evidência do estado DOWN; portanto, ele é mantido aqui como comportamento documentado da implementação, e não como resultado experimental comprovado pelas imagens.

3.4 TTL

TTL (Time To Live) define por quanto tempo uma recomendação permanece armazenada no Redis. No projeto, o valor configurado é de 300 segundos.

As evidências mostram esse mecanismo em funcionamento: na requisição que gerou MISS, o campo `ttl_remaining_seconds` é 300; na requisição seguinte, já classificada como HIT, o mesmo campo aparece com 295 segundos. Isso demonstra, dentro da sequência registrada nos prints, que a resposta em cache possui um tempo de expiração associado.

A Figura 13 também registra `ttl_seconds = 300` no endpoint de estatísticas, reforçando o valor configurado para o TTL.

3.5 Invalidação do Cache

Foi implementada a rota DELETE /cache para remover as recomendações armazenadas no cache. Na Figura 11, a chamada retorna código HTTP 200 e `keys_removed = 1`, indicando que a recomendação que estava armazenada foi removida.

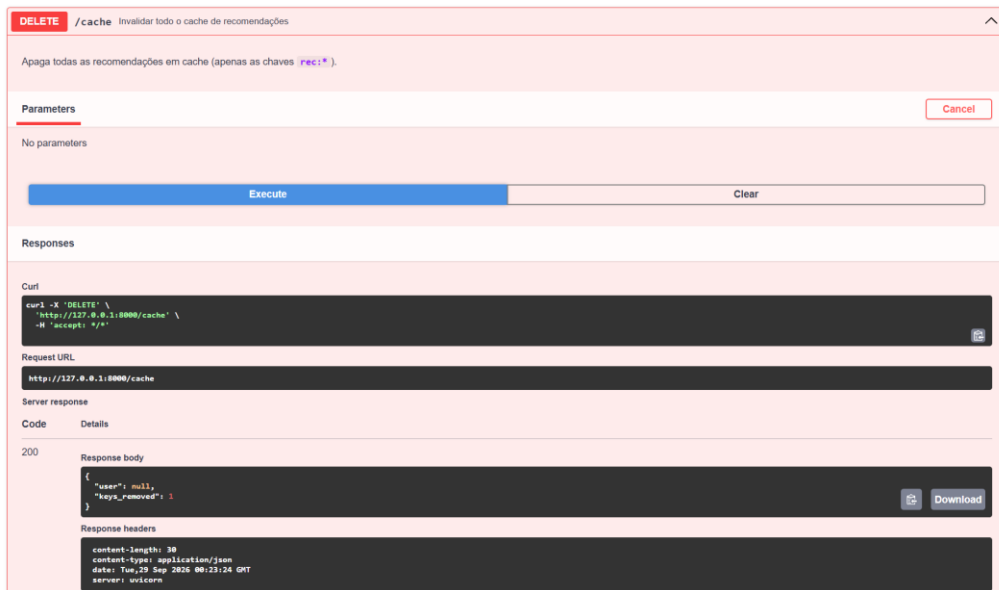


Figura 11 — Execução de DELETE /cache, com resposta HTTP 200 e keys_removed = 1.

A consulta às estatísticas realizada após a invalidação (Figura 13) mostra cached_recommendations = 0 e a lista de chaves vazia, o que é coerente com a remoção registrada na Figura 11.

Também foi executada a rota DELETE /cache/users/{nome}, que invalida seletivamente as recomendações de um único usuário. Na Figura 12, a chamada para a usuária Ana retorna HTTP 200 e keys_removed = 1.

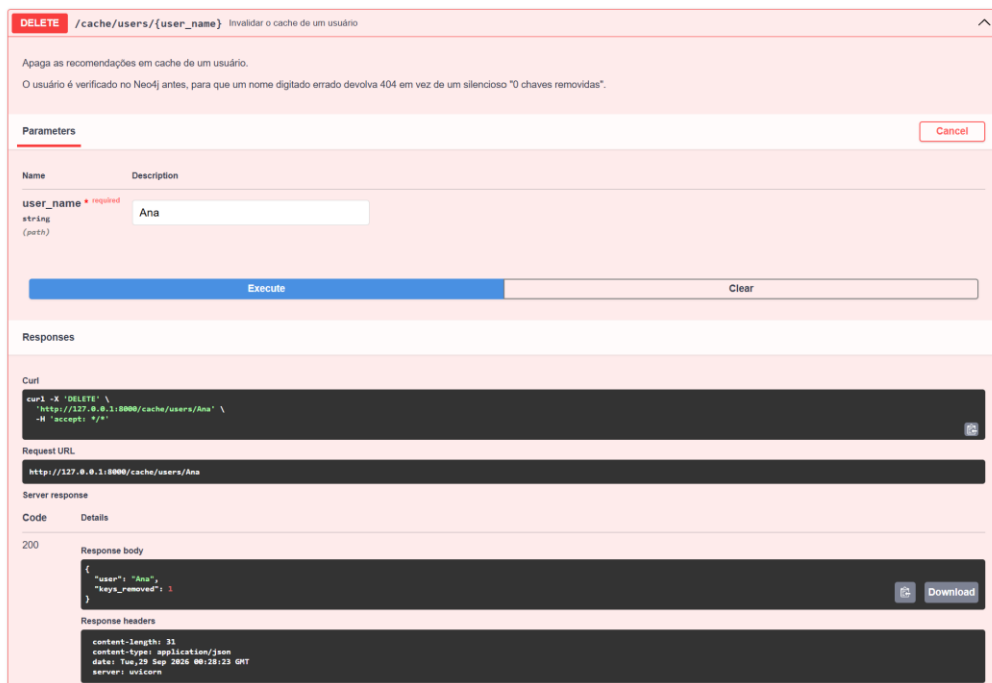


Figura 12 — Execução de DELETE /cache/users/Ana, com resposta HTTP 200 e keys_removed = 1.

3.6 Estatísticas

O endpoint GET /cache/stats foi criado para consultar o estado do cache. A Figura 13 apresenta uma execução com HTTP 200 e informa, entre outros campos,

cache_enabled = true, ttl_seconds = 300, redis = connected e
cached_recommendations = 0, com a lista de chaves vazia após a invalidação da
Figura 11.

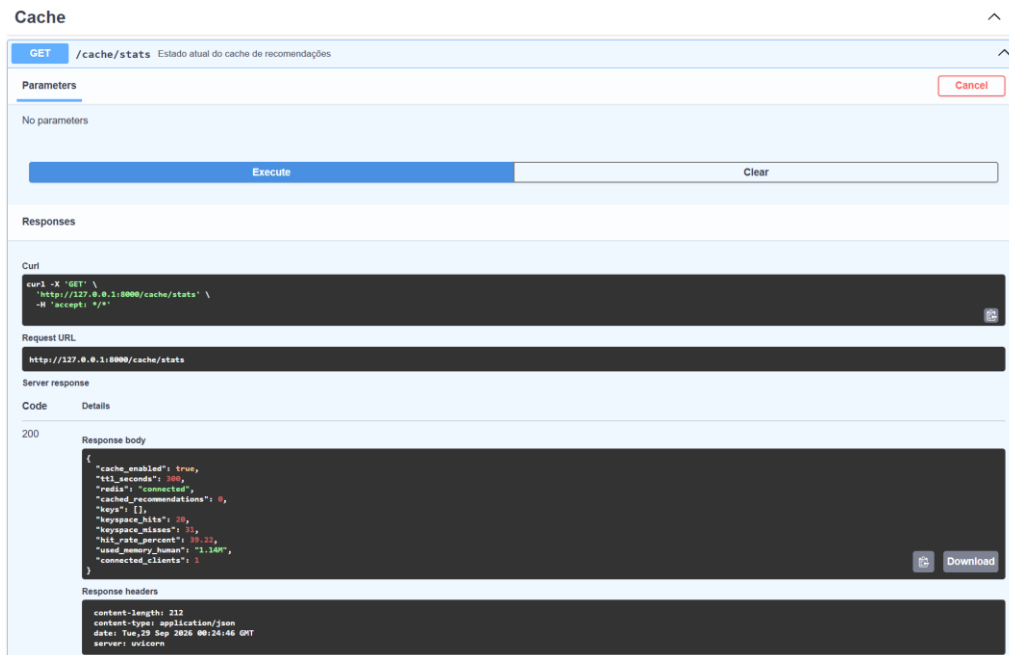


Figura 13 — Estatísticas do cache após a invalidação, evidenciando Redis conectado, cache habilitado, TTL de 300 segundos e nenhuma recomendação armazenada.

Na mesma resposta, o Redis informa `keyspace_hits = 20`, `keyspace_misses = 31` e `hit_rate_percent = 39,22`. Esses contadores são acumulados pelo servidor desde a inicialização do container e, por isso, não representam apenas o pequeno conjunto de requisições mostrado nas Figuras 9 e 10. Para uma análise específica do experimento, a taxa de acerto deve ser calculada a partir das respostas da própria API:

$$\text{Taxa de acerto} = \text{HIT} / \text{total de requisições} \times 100$$

Em uma nova consulta ao mesmo endpoint, feita em momento posterior (Figura 14), o Redis informa `cached_recommendations = 1` e a chave `rec:Julia:10`, evidência direta de que o cache identifica a recomendação pelo nome do usuário e pelo limite utilizado. Os contadores acumulados passam a `keyspace_hits = 24` e `keyspace_misses = 37` (`hit_rate_percent = 39,34`), valores coerentes com a evolução em relação à Figura 13.

Cache

GET /cache/stats Estado atual do cache de recomendações

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  http://127.0.0.1:8080/cache/stats' \
  -H 'accept: */*'
```

Request URL

http://127.0.0.1:8080/cache/stats

Server response

Code Details

200

Response body

```
{
  "cache_enabled": true,
  "ttl_seconds": 300,
  "redis": "connected",
  "cached_recommendations": 1,
  "keys": [
    "rec:Julia:10"
  ],
  "keyspace_hits": 26,
  "keyspace_misses": 37,
  "hit_rate_percent": 99.34,
  "used_memory_human": "1.34M",
  "connected_clients": 1
}
```

Response headers

```
content-length: 226
content-type: application/json
date: Tue, 29 Sep 2026 00:30:51 GMT
server: uvicorn
```

Figura 14 — Nova consulta ao GET /cache/stats, com a chave rec:Julia:10 armazenada e cached_recommendations = 1.

4. Testes de Desempenho

Após a integração do Redis, os testes de desempenho foram organizados para observar o comportamento da API com cache, comparando principalmente respostas classificadas como MISS e HIT. A rota utilizada é:

```
GET /users/{nome}/recommendations?limit=10
```

As evidências anexadas permitem comparar diretamente requisições MISS e HIT para três usuários (Julia, Carlos e Ana) e registram uma requisição com o cache desativado (DISABLED). Um cenário completo sem cache, com uma série de medições, não está comprovado pelos prints; por isso, a linha correspondente da tabela de resultados traz apenas a medição individual disponível.

4.1 Teste sem Cache

O documento-base define o cenário sem cache por meio da configuração `CACHE_ENABLED=false`. Nesse modo, as recomendações são obtidas diretamente pela consulta ao Neo4j, sem utilização do Redis para atender a resposta.

```
CACHE_ENABLED=false
```

Após reiniciar a API com o cache desativado, a requisição para a rota de recomendação retorna `cache = DISABLED`, como mostra a Figura 15. A resposta traz `response_time_ms = 9,96` e `ttl_remaining_seconds = null` (não há TTL, pois o Redis não é utilizado), com a mesma recomendação obtida nos demais testes: "O Senhor dos Anéis", com Carlos como referência, 2 filmes em comum e média de notas 4,5.

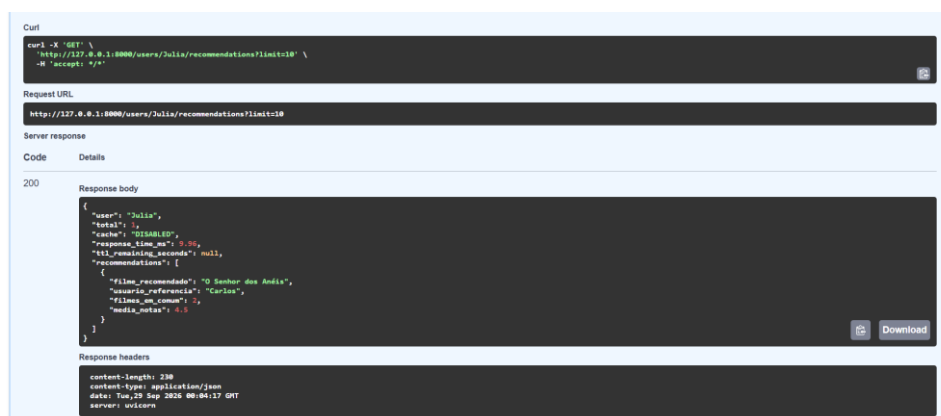


Figura 15 — Teste sem cache: requisição para Julia com `cache = DISABLED`, 9,96 ms e `ttl_remaining_seconds = null`.

Trata-se de uma única medição, sem série de requisições; por isso ela é registrada na tabela de resultados, mas não permite calcular uma média do cenário sem cache.

4.2 Teste com Cache

No cenário com cache, a sequência observada nos prints é compatível com o funcionamento esperado do cache-aside. A primeira requisição para Julia, registrada na Figura 16, resulta em MISS e apresenta 10 ms. A requisição subsequente, registrada na Figura 17, resulta em HIT e apresenta 1,64 ms.

A sequência comprovada pelas evidências (Figuras 16 e 17) pode ser representada da seguinte forma:

- 1ª requisição → MISS → consulta ao Neo4j → gravação no Redis
- 2ª requisição → HIT → retorno pelo Redis

As imagens não mostram uma captura direta da consulta ao Neo4j; a participação do Neo4j no MISS decorre da estratégia de cache descrita na documentação do endpoint e do estado MISS retornado pela API.

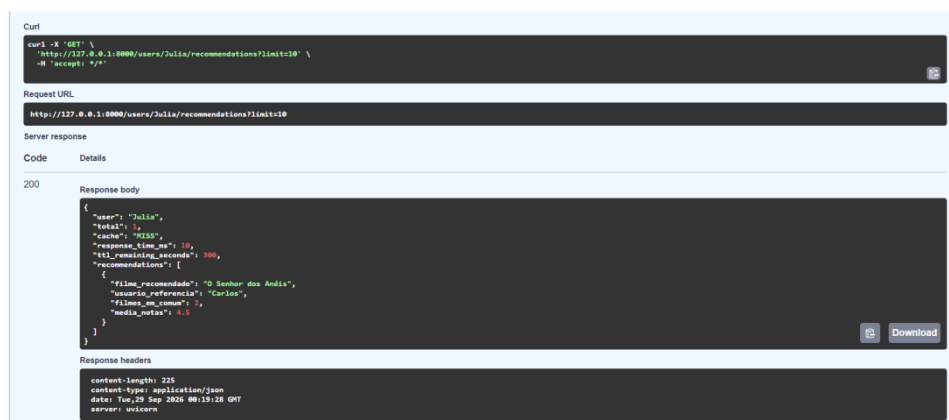


Figura 16 — Teste com cache, 1ª requisição para Julia: CACHE MISS, 10 ms e TTL restante de 300 s.

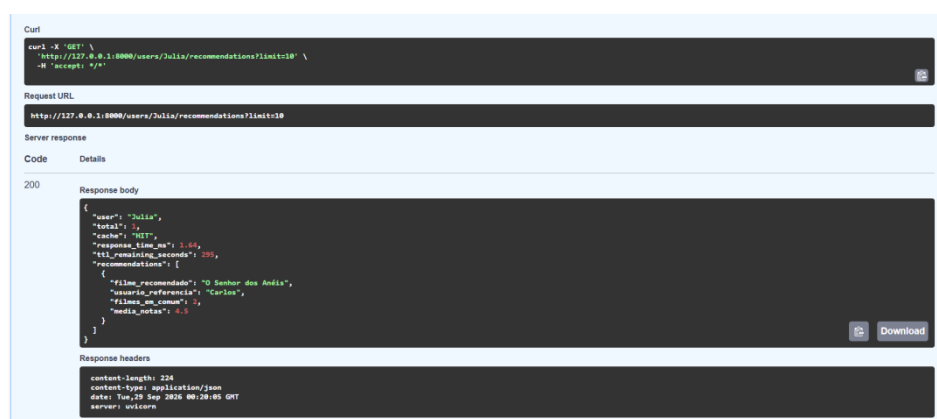


Figura 17 — Teste com cache, 2ª requisição para Julia: CACHE HIT, 1,64 ms e TTL restante de 295 s.

4.3 Aquecimento da API

O documento-base registra que a primeira requisição após o início da API pode apresentar tempo superior às demais devido à abertura da conexão com o Neo4j e recomenda aquecer a aplicação antes de calcular médias.

Os prints anexados nesta etapa não comprovam uma requisição inicial de aproximadamente 1200 ms nem uma série suficiente para validar as faixas de 20–25 ms para MISS e 3–5 ms para HIT. Portanto, esses valores não são tratados como resultado do experimento anexado. As medições individuais comprovadas pelas imagens são: MISS de 10 ms (Julia), 12,91 ms (Carlos) e 9,27 ms (Ana); HIT de 1,64 ms (Julia), 1,45 ms (Carlos) e 1,43 ms (Ana).

4.4 Taxa de acerto

A taxa de acerto deve ser calculada a partir das respostas efetivamente classificadas como HIT:

$$\text{Taxa de acerto} = \text{HIT} / \text{total de requisições} \times 100$$

Considerando as seis requisições comprovadas nas Figuras 16 a 19 (uma requisição MISS e uma HIT para cada um dos três usuários), há 3 HIT em 6 requisições observadas:

$$\text{Taxa de acerto} = 3 / 6 \times 100 = 50\%$$

Como a sequência foi montada com um MISS e um HIT por usuário, o valor de 50% decorre do próprio desenho do teste e não representa a taxa esperada em uso contínuo. Esse percentual também não deve ser confundido com o `hit_rate_percent = 39,22` apresentado pelo Redis na Figura 13, pois o contador do Redis é acumulado pelo servidor desde a inicialização do container.

Os estados DOWN e DISABLED, quando utilizados pela aplicação, não devem ser contabilizados como HIT ou MISS.

4.5 Redução das consultas ao Neo4j

O cache tem como objetivo reduzir consultas repetidas ao Neo4j quando a mesma recomendação já está armazenada no Redis. Na sequência comprovada para Julia, Carlos e Ana, a primeira requisição de cada usuário é MISS e a seguinte é HIT. Dessa forma, a segunda resposta de cada par foi classificada pela API como HIT, indicando que o resultado já estava disponível na camada de cache.

Não há, nos prints anexados, um contador de consultas ao banco. Por isso, a redução é apresentada como efeito funcional do mecanismo: segundo a descrição do endpoint exibida nos prints, uma resposta HIT não consulta o Neo4j, de modo que 3 das 6 requisições observadas (as HIT) dispensaram essa consulta.

Para uma sequência de 10 solicitações, o documento-base apresenta como exemplo teórico a possibilidade de ocorrer 1 MISS e 9 HIT; esse exemplo não deve ser interpretado como resultado medido nos prints anexados.

4.6 Teste com diferentes usuários

O projeto utiliza o nome do usuário na própria URL da requisição, permitindo separar as recomendações por usuário. O documento-base considera Julia, Carlos e Ana para esse teste.

Os testes foram executados para os três usuários, cada um com uma requisição MISS seguida de uma requisição HIT. Para Julia, os resultados são os das Figuras 16 e 17 (MISS de 10 ms e HIT de 1,64 ms). Para Carlos, a Figura 18 mostra MISS de 12,91 ms e HIT de 1,45 ms; para Ana, a Figura 19 mostra MISS de 9,27 ms e HIT de 1,43 ms.

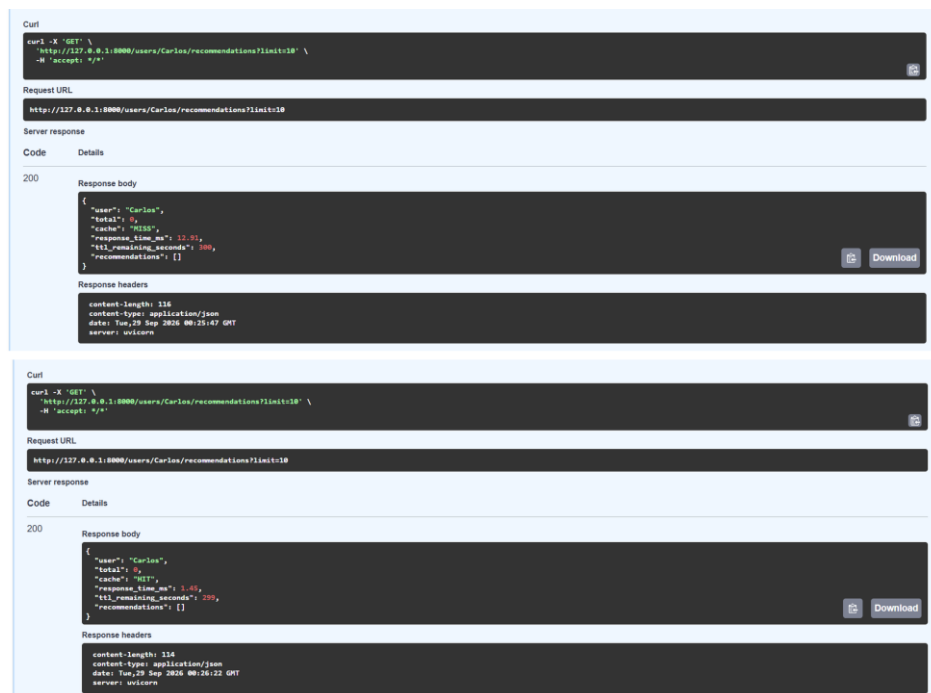


Figura 18 — Requisições para Carlos: 1ª com CACHE MISS (12,91 ms) e 2ª com CACHE HIT (1,45 ms).

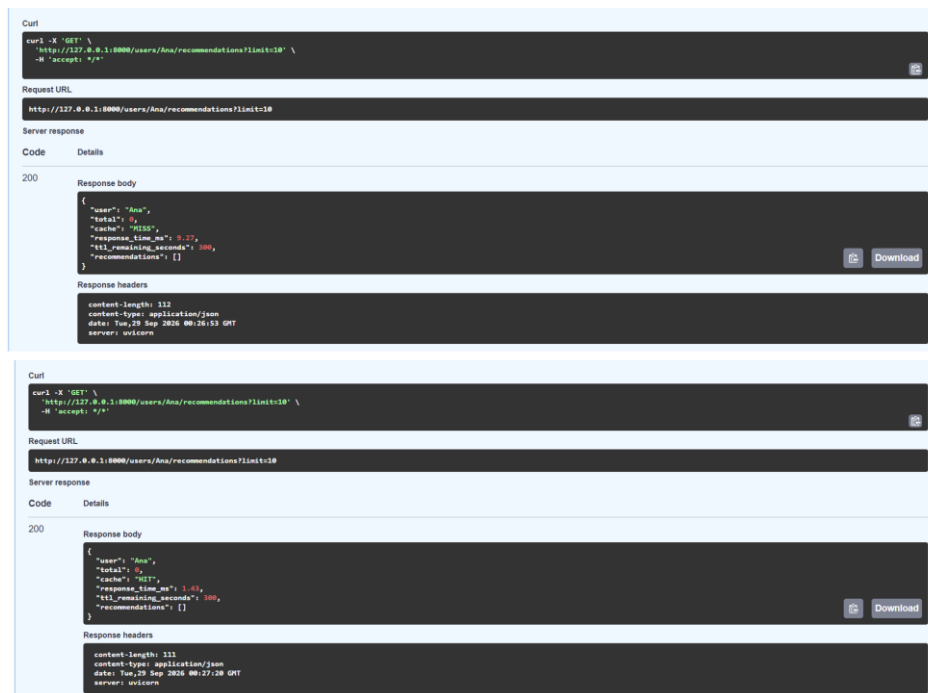


Figura 19 — Requisições para Ana: 1ª com CACHE MISS (9,27 ms) e 2ª com CACHE HIT (1,43 ms).

Nas respostas de Carlos e de Ana, o campo total é 0 e a lista de recomendações é vazia, ou seja, não há filme a recomendar para esses dois usuários com os dados cadastrados. Mesmo assim, o resultado foi armazenado no cache: a primeira requisição de cada um foi MISS e a segunda, HIT. Além disso, a chamada de DELETE /cache/users/Ana removeu uma única chave (Figura 12) depois de as recomendações de Carlos e de Ana terem sido armazenadas, o que é compatível com o armazenamento independente por usuário. A chave rec:Julia:10 exibida na Figura 14 reforça que o cache é identificado pelo nome do usuário e pelo limite.

4.7 Considerações sobre o TTL

O TTL observado no experimento é de 300 segundos. Na Figura 9, o MISS retorna `ttl_remaining_seconds = 300`; na Figura 10, o HIT retorna `ttl_remaining_seconds = 295`. Nas requisições de Carlos e de Ana (Figuras 18 e 19), os MISS retornam 300 e os HIT retornam 299 e 300, respectivamente. A Figura 13 registra `ttl_seconds = 300` no estado do cache.

Esses valores mostram a presença do TTL nas respostas, mas os prints não acompanham a expiração completa de uma chave após 300 segundos. Portanto, não é possível afirmar, a partir dessas evidências, que uma requisição posterior à expiração tenha sido efetivamente registrada como novo MISS.

Para reproduzir um teste de expiração, é necessário registrar a configuração utilizada, aguardar o período definido e executar novamente a mesma requisição, documentando o novo estado retornado pela API.

4.8 Análise dos resultados

As medições disponíveis permitem observar uma diferença de tempo entre MISS e HIT para os três usuários testados: 10 ms contra 1,64 ms para Julia, 12,91 ms contra 1,45 ms para Carlos e 9,27 ms contra 1,43 ms para Ana. Considerando os três MISS e os três HIT, o tempo médio é de 10,73 ms nas respostas MISS e de 1,51 ms nas respostas HIT; a média dos HIT corresponde a cerca de 14% da média dos MISS, uma redução de cerca de 86%.

Para o conjunto das seis requisições, com 3 HIT e 3 MISS, o tempo médio aritmético é de 6,12 ms. Esse valor é apresentado apenas como média das seis observações anexadas e não como média geral de desempenho da API.

A única medição sem cache (9,96 ms, Figura 15) fica na mesma faixa dos MISS (9,27 a 12,91 ms), o que é esperado, já que ambos consultam o Neo4j, e bem acima dos HIT (1,43 a 1,64 ms). O resultado é compatível com o objetivo do cache: uma resposta já armazenada pode ser retornada sem repetir o processamento necessário para obtê-la. Entretanto, cada valor corresponde a uma única medição, e os prints não permitem medir diretamente a quantidade de consultas ao Neo4j nem estabelecer uma média estatisticamente representativa de uma carga maior.

Tabela de resultados observados

Cenário	Requisições	HIT	MISS	Tempo médio
Cache habilitado — MISS (Julia, Carlos e Ana)	3	0	3	10,73 ms
Cache habilitado — HIT (Julia, Carlos e Ana)	3	3	0	1,51 ms
Sequência observada — total	6	3	3	6,12 ms
Sem cache (DISABLED) — Julia	1	—	—	9,96 ms

Tabela 1 — Resultados comprovados pelas evidências anexadas (médias aritméticas de medições individuais); no cenário sem cache há apenas uma medição, e os estados DISABLED não são contados como HIT ou MISS.

4.9 Documentação da execução

Para reproduzir a execução, devem ser iniciados os containers do projeto e a API FastAPI. Em seguida, deve ser verificada a disponibilidade da API por meio de GET /health.

Para o teste com cache, recomenda-se limpar as recomendações armazenadas antes da medição:

```
DELETE /cache
```

Depois, deve ser executada a rota de recomendações para o usuário e limite desejados:

```
GET /users/{nome}/recommendations?limit=10
```

Em cada execução, devem ser registrados pelo menos os campos cache, response_time_ms e ttl_remaining_seconds. Para estatísticas gerais do cache, pode ser utilizado:

```
GET /cache/stats
```

A documentação anexada comprova a sequência de execução para Julia, Carlos e Ana, incluindo MISS, HIT, TTL, invalidação (geral e por usuário) e consulta às estatísticas. Para completar um experimento comparativo formal entre cache habilitado e desabilitado, ainda seria necessário registrar uma série de requisições com CACHE_ENABLED=false e outra série com CACHE_ENABLED=true, mantendo as mesmas condições de execução e preenchendo a tabela com os valores efetivamente obtidos.